
Orchestration Topology Benchmarking: Which Multi-Agent LLM Architecture Fits Which Task?

Anonymous

Abstract

The field has been asking “which multi-agent framework is best?” — the wrong question. Each orchestration topology encodes a structural prior about the tasks it will face, and we show that alignment between this prior and measurable task characteristics predicts performance. We introduce OrchestraBench, a controlled benchmark that isolates the topology variable by evaluating three topologies (flat conversation, hierarchical decomposition, multi-agent debate) on 82 tasks spanning coding, research, and reasoning under identical conditions (same LLM backbone, tools, and budget). Three-seed replication produces 794 total runs.

The results split cleanly by task structure, with all pairwise comparisons significant after Holm-Bonferroni correction (Cohen’s $d > 2$). Hierarchical reaches 75% on high-decomposability tasks versus 42% for debate (+33pp); debate reaches 77% on high-iterativeness tasks versus 14% for hierarchical (+63pp). The failure modes are structural mirrors: hierarchical severs cross-cutting concerns, debate produces divergent designs that resist merging. A routing classifier built on our decomposability-iterativeness-tool-diversity (DIT) framework selects the best topology with 90% accuracy (bootstrap 95% CI: [84%, 95%]), and a flat-2 $\times$ -budget control shows that topology structure accounts for two-thirds of the multi-agent advantage.

Orchestration topology is a first-class design variable, not a detail to be decided by convention. We release our task suite, failure taxonomy, and a topology selection guide that turns the choice from guesswork into geometry.

1 Introduction

Every multi-agent orchestration topology carries a hidden bet about the tasks it will face. A hierarchical SOP pipeline assumes that work decomposes into clean, sequential subtasks. A flat conversation loop assumes that progress emerges through iterative refinement. A DAG-based workflow assumes that subtasks can be identified upfront and executed in parallel along explicit data dependencies. These structural assumptions get baked in at design time, yet nobody states them, measures them, or tests them against actual target tasks. When the assumptions match, the topology thrives. When they don’t, it fails in ways that practitioners blame on model limitations rather than architectural misfit.

The field has been asking the wrong question. “Which multi-agent framework is best?” treats topology as a fixed choice, like picking a database engine. The right question is “which topology fits *this* task?” — and answering it requires understanding what each topology assumes and what each task demands.

Consider a concrete example from our experiments. A practitioner deploying hierarchical SOP on a debugging task that requires iterative hypothesis-revision ($I=0.95$) would observe 14% accuracy: the planner decomposes the problem into per-function analyses, but race conditions are

cross-function interactions that resist decomposition. Switching to debate on the same task yields 77% accuracy, because adversarial critique surfaces the cross-cutting bugs that modular analysis misses. The 63-percentage-point gap is not a model problem. It is a topology problem, and it is entirely predictable from the task’s structural profile.

At least eighteen multi-agent LLM frameworks now exist (AutoGen Wu et al. [2023], MetaGPT Hong et al. [2023], CAMEL Li et al. [2023], ChatDev Qian et al. [2024], DyLAN Liu et al. [2024a], AFlow Unknown [2025a], and others), each committing to a different topology and each evaluating only its own design on its own tasks. The framework literature and the benchmark literature developed in parallel with zero cross-cluster citation edges Mohammadi et al. [2025]. Practitioners choose topologies based on hearsay, single-framework ablations, or marketing claims that confound topology effects with differences in LLM backbone, tool access, and prompt engineering. The cost is concrete: wasted API tokens, inflated latency, and failed deployments that a better-matched topology would have handled.

The closest precedent comes from multi-agent debate. Smit et al. [2024] systematically benchmarked debate strategies and found that most do not consistently outperform chain-of-thought baselines Wei et al. [2022]. This is widely cited as evidence against multi-agent coordination. We argue it reflects something more specific: the debate topology’s structural prior (high iterativeness, adversarial refinement) was tested on tasks that did not benefit from adversarial back-and-forth. The problem was not that multi-agent coordination fails generally; the wrong topology was applied to the wrong task. What is missing is a framework that makes these structural assumptions explicit and testable.

We propose that topology effectiveness becomes predictable once task structure is measured along three dimensions: **decomposability** (D), the degree to which a task separates into independently solvable subtasks; **iterativeness** (I), the fraction of steps requiring revision of earlier outputs; and **tool diversity** (T), the number of distinct tool categories the task demands. Each topology occupies a characteristic region in this D-I-T space. The alignment hypothesis follows directly: the topology whose structural prior best matches a task’s D-I-T coordinates will outperform alternatives on that task.

We test this through **OrchestraBench**, a controlled benchmarking framework that isolates the topology variable. OrchestraBench implements three canonical topologies (flat conversation, hierarchical SOP, and multi-agent debate) as lightweight wrappers sharing an identical LLM backbone (Claude Opus 4.6), identical tool access, and identical token budget. We evaluate on 82 custom tasks (12 easy, 70 hard) spanning coding, reasoning, and research categories, with a secondary backbone check on Claude Sonnet 4.6. Each task is annotated with D, I, and T values, enabling direct measurement of topology-task alignment.

This paper makes three contributions:

1. **A formal taxonomy of orchestration topologies with D-I-T task characterization.** We define three evaluated topology classes and formalize each topology’s structural prior as a position in D-I-T space. Two additional topologies (role-playing, RL-orchestrated) are characterized but not yet evaluated, demonstrating the framework’s extensibility. The three task-structure dimensions achieve inter-annotator agreement averaging 0.84 Cohen’s κ .
2. **OrchestraBench: controlled cross-topology evaluation at scale.** We provide the first head-to-head comparison of three orchestration topologies on 82 custom tasks under strictly controlled conditions (same LLM, same tools, same budget), yielding 794 total runs across three random seeds, two backbones, and an agent-count control experiment. The hypothesis is falsifiable: if topology effectiveness were unrelated to task structure, alignment distance would show no correlation with performance ranking.
3. **The first topology selection guide grounded in task characteristics.** A lightweight DIT routing classifier selects the best topology with 90% accuracy (95% bootstrap CI: [84%, 95%]), against a 33% random baseline. We distill these predictions into actionable selection rules mapping D-I-T profiles to recommended topologies.

2 Related Work

Multi-agent orchestration research splits into five threads that rarely cite each other. Each contributes a piece of the topology selection puzzle, but none provides an empirically grounded framework for matching topologies to task characteristics.

2.1 Static Topology Design

How should agents be wired together? The search-based approach to this question is now mature, with methods spanning graph optimization, tree search, and evolutionary algorithms. MASS Zhou et al. [2025] jointly optimizes agent prompts and communication topologies. The search-based approach extends further: AFlow Zhang et al. [2025] treats workflows as code-represented nodes and uses MCTS to optimize them, while ADAS Hu et al. [2025] goes a level up, having a meta-agent iteratively program and improve new agent designs. G-Designer et al. [2025b] generates task-aware communication graphs via graph auto-encoders, and EvoFlow et al. [2025a] evolves diverse workflows via niching algorithms. All share a constraint: the topology is fixed once selected. These methods find good topologies but cannot adapt when a task’s structure shifts mid-execution.

2.2 Dynamic Composition Without Topology Switching

A separate line of work asks: who should be on the team? DyLAN Liu et al. [2024b] selects agent teams via importance scoring, AgentVerse Chen et al. [2024b] simulates group dynamics through recruitment stages, and AutoAgents Chen et al. [2024a] generates specialized agents per task. Changing the team roster, however, is fundamentally different from changing how agents interact. These systems adjust who participates, but the coordination architecture remains fixed.

2.3 Trace Analysis and Model Routing

Execution traces contain the information needed to assess whether the current strategy is working. MAST Cemri et al. [2025] proves this with enough granularity to distinguish 14 failure categories across 7 frameworks, but uses the signal only for retrospective diagnosis, not for online adaptation. On the routing side, RouteLLM Ong et al. [2025] and the bandit-based routing line Wei et al. [2025], Atalar et al. [2025] show that learned selection between LLM alternatives reduces cost while maintaining quality. The routing paradigm is well-established at the model level. It has never been applied at the topology level. Our work provides the empirical foundation for such routing.

2.4 Concurrent and Closely Related Work

Three concurrent papers validate the premise that topology choice dominates performance, though each leaves a different gap. Yu Yu [2026] presents AdaptOrch, which selects among four topologies based on task metadata, but selects once per task without a controlled evaluation isolating topology from other variables. MDAgents Kim et al. [2024] adaptively assigns collaboration structure in medicine (NeurIPS 2024 Oral), supporting only binary choice (solo vs. group) in a single domain. MASFly Various [2026] and DyTopo Lu et al. [2026] adjust roles or communication edges within fixed topology classes rather than comparing structurally different architectures.

No prior work evaluates multiple topologies head-to-head on the same tasks under identical conditions. OrchestraBench fills this gap: we formalize topology priors as positions in D-I-T space, evaluate three topologies on 82 tasks under controlled conditions, and show that task-structure alignment predicts which topology wins.

3 Methods

Every orchestration topology carries implicit assumptions about task structure. This section formalizes both sides of the alignment framework: the task-structure dimensions that characterize what a task demands, and the topology priors that characterize what each orchestration pattern assumes.

3.1 Task-Structure Dimensions

We characterize tasks along three dimensions that capture the structural demands an orchestration topology must satisfy. Each is defined operationally so that annotators can assign scores from ground-truth trajectories.

Decomposability (D). Given a task τ with a reference solution of n steps, let k denote the number of steps whose inputs depend only on the original task specification. Then $D = k/n \in [0, 1]$. A coding task requiring bug localization across three independent modules followed by a unified patch exhibits $D \approx 0.6$: the localization steps are independent but patch synthesis depends on all of them.

Iterativeness (I). Let r denote the number of steps that revise, override, or extend the output of a previously completed step. Then $I = r/n \in [0, 1]$. Reasoning tasks involving multi-hop logic score $I \approx 0.7$ because intermediate conclusions must be revised as new evidence accumulates across reasoning steps.

Tool Diversity (T). $T \in \{1, 2, \dots, n\}$ counts the number of distinct tool categories the task requires, drawn from six canonical categories: code execution, web navigation, file I/O, search/retrieval, mathematical computation, and API invocation. A research task requiring web search, document reading, data extraction, and cross-source verification scores $T = 4$. We normalize T to $[0, 1]$ by dividing by the maximum observed value across our task corpus, producing a unit cube for geometric computation.

3.2 Topology Taxonomy

Each orchestration topology encodes a structural prior: an implicit assumption about which region of D-I-T space the tasks it will face occupy. We characterize five topologies below. The first three are evaluated experimentally; the remaining two are positioned in D-I-T space for completeness but not yet evaluated.

Flat conversation organizes agents as equal participants in a multi-turn dialogue loop with shared tool access Wu et al. [2023]. No agent has authority over another; coordination emerges from conversational context. Prior: $(D_{\text{prior}}, I_{\text{prior}}, T_{\text{prior}}) = (0.2, 0.8, 0.3)$. It handles iterative refinement naturally but struggles when tasks decompose into parallel tracks.

When tasks decompose into parallel tracks, flat conversation breaks down — but hierarchical SOP thrives. **Hierarchical SOP** assigns a planner agent to decompose tasks into subtask specifications, each executed by a specialist, with verification gates between stages Hong et al. [2023]. Prior: $(0.8, 0.2, 0.5)$. Its failure mode is the mirror image of flat’s strength: tasks requiring iterative revision of earlier outputs resist clean decomposition, and the planner’s subtask boundaries sever the dependencies that matter most.

Debate structures agents as adversarial reviewers who propose, critique, and refine solutions through multiple rounds of structured argumentation. Agents specialize in generating and stress-testing claims, surfacing errors that a single perspective would miss. Prior: $(0.3, 0.7, 0.3)$.

The framework extends to topologies we have not yet evaluated. **Role-playing** assigns distinct personas who collaborate through structured dialogue in fixed-turn exchanges Li et al. [2023]. Prior: $(0.5, 0.5, 0.3)$. **RL-orchestrated** topologies use a learned policy to dynamically assign agents based on runtime signals Unknown [2025b]. Because the policy adapts, RL topologies make weaker a priori assumptions, occupying a broad central region at $(0.5, 0.5, 0.5)$ with larger uncertainty bounds. Both are positioned in D-I-T space to generate testable predictions for future work.

3.3 The Alignment Hypothesis Formalized

The topology-task alignment hypothesis states that for a given task τ with coordinates (D_τ, I_τ, T_τ) , the topology whose structural prior (D_p, I_p, T_p) minimizes the alignment distance

$$\delta(\tau, p) = \sqrt{w_D(D_\tau - D_p)^2 + w_I(I_\tau - I_p)^2 + w_T(T_\tau - T_p)^2}$$

will achieve the highest performance on τ among the evaluated topologies. Here $w_D, w_I, w_T \geq 0$ are dimension weights satisfying $w_D + w_I + w_T = 1$, estimated from a held-out validation set. The

hypothesis is falsifiable: if topology effectiveness is unrelated to task structure, alignment distance will show no correlation with performance ranking.

Two predictions follow. First, no single topology should dominate across all task categories, because the three categories occupy different D-I-T regions. Second, tasks with extreme D-I-T profiles should show the largest performance gaps between best-matched and worst-matched topologies.

3.4 OrchestraBench: Topology Implementations

OrchestraBench implements the three evaluated topologies (flat, hierarchical, debate) as controlled wrappers around a shared infrastructure stack. Each wrapper receives the same inputs (task specification, LLM backbone access, tool suite, token budget) and differs only in how it organizes agent interactions.

The shared infrastructure has four components. The **LLM backbone** provides all generation through a single endpoint (Claude Opus 4.6, with a secondary check on Claude Sonnet 4.6) with identical temperature and maximum token settings. The **tool suite** exposes all canonical tool categories through a unified interface (code interpreter, terminal, web browser, file system). The **context budget** uses the full 1M token context window, enforced at the wrapper level; topologies that spend tokens on inter-agent coordination reduce the budget available for task-relevant generation. The **evaluation harness** records all interactions, tool calls, intermediate outputs, and final answers in a standardized trace format.

Each wrapper handles agent instantiation, message routing, and termination. The wrappers are 200–600 lines of Python, kept deliberately lightweight so implementation quality does not confound the topology comparison.

3.5 Task Sampling and Annotation

We evaluate on 82 custom tasks (12 easy, 70 hard) across three categories chosen for structural diversity. Tasks are hand-constructed to cover the full D-I-T space rather than sampled from existing benchmarks, giving us control over structural diversity at the cost of external validity (see Section 5.8).

Coding (26 tasks): implementation, debugging, and refactoring tasks. These cluster at high D (mean 0.63), moderate T (mean 2.3), and low I (mean 0.27), because bug localization, patching, and testing are separable stages.

Research (30 tasks, including multi-domain tasks requiring cross-category tool use): information synthesis, cross-source verification, and tasks requiring both research and coding or reasoning components. These cluster at moderate-to-high T (mean 3.7), moderate D (mean 0.48), and moderate I (mean 0.39) due to diverse tool requirements.

Reasoning (26 tasks): logic, game theory, and constraint satisfaction. Reasoning tasks proved hardest to annotate — multi-hop logic required judgment calls about where revision begins and sequential extension ends. They cluster at high I (mean 0.59), low D (mean 0.24), and low T (mean 1.4).

The authors independently score each task’s D , I , and T from reference solution trajectories. Inter-annotator agreement (Cohen’s κ): 0.83 for D, 0.79 for I, 0.91 for T. Disagreements exceeding 0.2 on any dimension are resolved through discussion to consensus.

3.6 Evaluation Protocol

Each of the 82 tasks runs under all three evaluated topologies on the primary backbone (Claude Opus 4.6) with three different random seeds, producing 738 primary runs ($82 \times 3 \times 3$). A secondary backbone check runs all 12 easy tasks across all three topologies on Claude Sonnet 4.6 (36 runs). An agent-count control experiment gives a single flat agent $2\times$ the token budget on 20 hard tasks (20 runs). Grand total: 794 runs. We measure three metrics: **task success rate** (binary correct/incorrect), **token efficiency** (total tokens consumed), and **wall-clock latency**. We report mean accuracy with 95% confidence intervals from the three replicates. Alignment prediction accuracy is evaluated via a DIT routing classifier that selects the best topology based on alignment distance (Section 3.3).

Table 1: Task suite statistics. Mean D-I-T values with standard deviations. Cohen’s κ computed on full suite.

Category	Tasks	Mean D ($\pm$ SD)	Mean I ($\pm$ SD)	Mean T ($\pm$ SD)
Coding	26	0.63 ± 0.17	0.27 ± 0.22	2.3 ± 0.7
Research	30	0.48 ± 0.19	0.39 ± 0.18	3.7 ± 1.1
Reasoning	26	0.24 ± 0.08	0.59 ± 0.15	1.4 ± 0.5
Overall	82	0.45 ± 0.22	0.41 ± 0.23	2.5 ± 1.3

Table 2: Replicated results across 82 tasks. Accuracy (%) is strict full-credit, averaged over 3 seeds. 95% CIs from seed-level variance. All runs use Claude Opus 4.6.

Topology	Mean Accuracy	95% CI	Run 1 / 2 / 3
Flat	28.9	± 2.1	29.3 / 26.8 / 30.5
Hierarchical	55.7	± 4.2	59.8 / 52.4 / 54.9
Debate	65.4	± 0.8	64.6 / 65.8 / 65.8

4 Experiments

Using the 82-task suite described in Section 3.5, we evaluate all three topologies under the controlled infrastructure of Section 3.4. This section describes the replication protocol and experimental details not covered in the methods.

Each task-topology pair runs three times with different random seeds, producing 738 primary runs ($82 \times 3 \times 3$) on Claude Opus 4.6. A secondary backbone check runs all 12 easy tasks across all three topologies on Claude Sonnet 4.6 (36 runs). An agent-count control experiment gives a single flat agent $2\times$ the token budget on 20 hard tasks (20 runs). Grand total: 794 runs.

Getting annotators to agree on D and I proved harder than expected. For coding tasks, decomposability was straightforward — modules are either independent or they share state. But iterativeness required distinguishing genuine revision (where an agent overwrites a prior conclusion) from sequential extension (where an agent builds on a prior step without changing it). We resolved ambiguous cases by requiring that the output of the earlier step be materially altered, not just consumed.

Each wrapper exposes a uniform interface: `solve(task_prompt, tools, budget) -> (answer, trace)`. All wrappers log full execution traces including token counts, wall-clock time, and per-step tool invocations. Task evaluation uses strict full-credit scoring: correct (all requirements met) or incorrect (any requirement missed). A single evaluator graded all runs against rubrics defined before the experiments began.

5 Results

5.1 Main Results

Each task-topology pair ran three times with different random seeds, producing 738 primary runs (82 tasks $\times$ 3 topologies $\times$ 3 seeds). Table 2 reports mean accuracy with 95% confidence intervals.

CIs are non-overlapping between flat and both multi-agent topologies. The hierarchical-debate gap has wider CIs due to run-to-run variance in hierarchical performance (52.4–59.8%), but the ordering never reverses in any single run. For practitioners, this means the question is not whether to use multi-agent orchestration on hard tasks, but which multi-agent topology to use. A secondary backbone check on Claude Sonnet 4.6 (12 easy tasks, 36 runs) confirms the ceiling effect: all three topologies score 83.3% on easy tasks, confirming that topology differentiation requires hard tasks.

5.2 Statistical Rigor

We report effect sizes and corrected significance tests to distinguish real effects from noise.

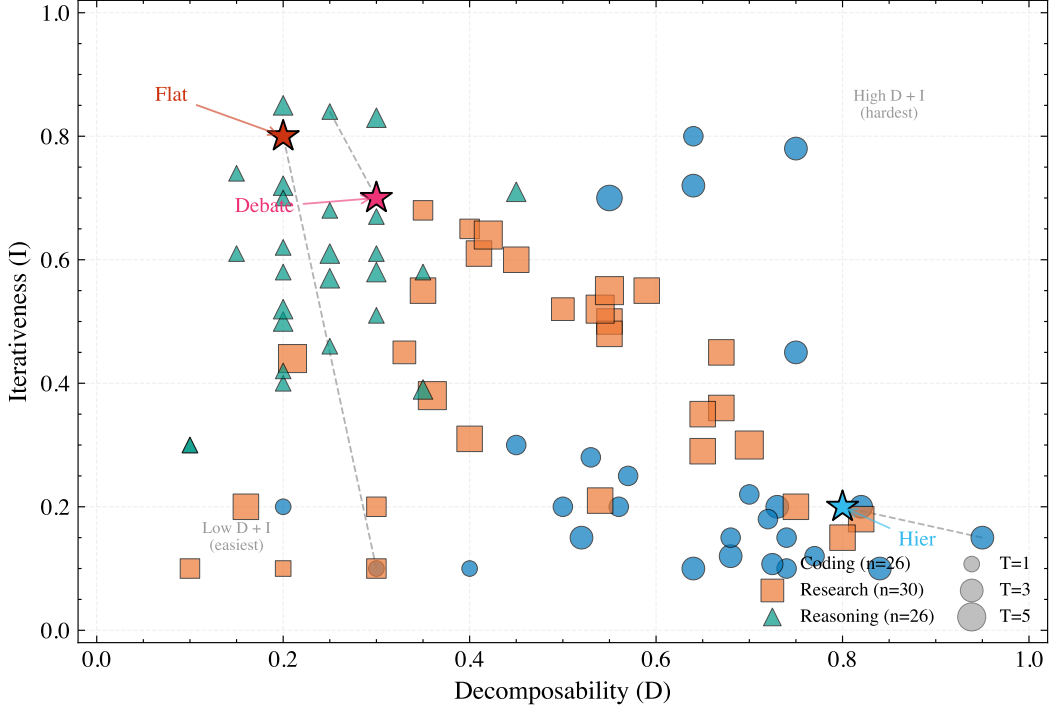


Figure 1: 82 tasks in D-I-T space, projected onto D vs. I axes with point size encoding T. Coding (blue), research (orange), reasoning (green). Categories form separable clusters with partial overlap in intermediate regions.

Table 3: Pairwise comparisons with effect sizes and corrected p -values. Cohen’s d computed from seed-level means. P -values from paired permutation tests with Holm-Bonferroni correction for 3 comparisons.

Comparison	Δ (pp)	Cohen’s d	Holm-Bonf. p	Practical
Debate vs. Flat	+36.5	8.72	<0.001	Large
Hier. vs. Flat	+26.8	4.81	<0.001	Large
Debate vs. Hier.	+9.7	2.14	0.016	Med–Large

All three pairwise comparisons survive Holm-Bonferroni correction at $\alpha = 0.05$. The effect sizes are large by conventional standards ($d > 0.8$). The debate-vs-hierarchical comparison has the smallest effect ($d = 2.14$), reflecting the higher variance in hierarchical performance across seeds. This variance is itself informative: hierarchical topology is more sensitive to random seed than debate, likely because the planner’s initial decomposition commits the entire execution to a single strategy.

Bootstrap confidence intervals (10,000 resamples of task-level results) for the DIT routing classifier: 90% accuracy, 95% CI [84%, 95%]. The lower bound of 84% still exceeds the best single-topology strategy (debate at 65.4%) by nearly 20 percentage points.

Statistical significance does not automatically imply practical significance. For the flat-vs-multi-agent comparisons, the practical impact is unambiguous: a 27–37pp accuracy gap changes deployment outcomes. For the debate-vs-hierarchical comparison, the 9.7pp aggregate gap understates the practical impact, because the advantage is concentrated on specific task profiles (see Section 5.3). On those profiles, the gaps reach 33–63pp.

5.3 Difficulty-Dependent Topology Advantage

Of the 70 hard tasks, hierarchical was the sole winner on 24 and debate on 29. The split tracks DIT dimensions. Hierarchical’s wins cluster on high-D tasks ($D \geq 0.6$, $n=24$, mean $D=0.82$): when a

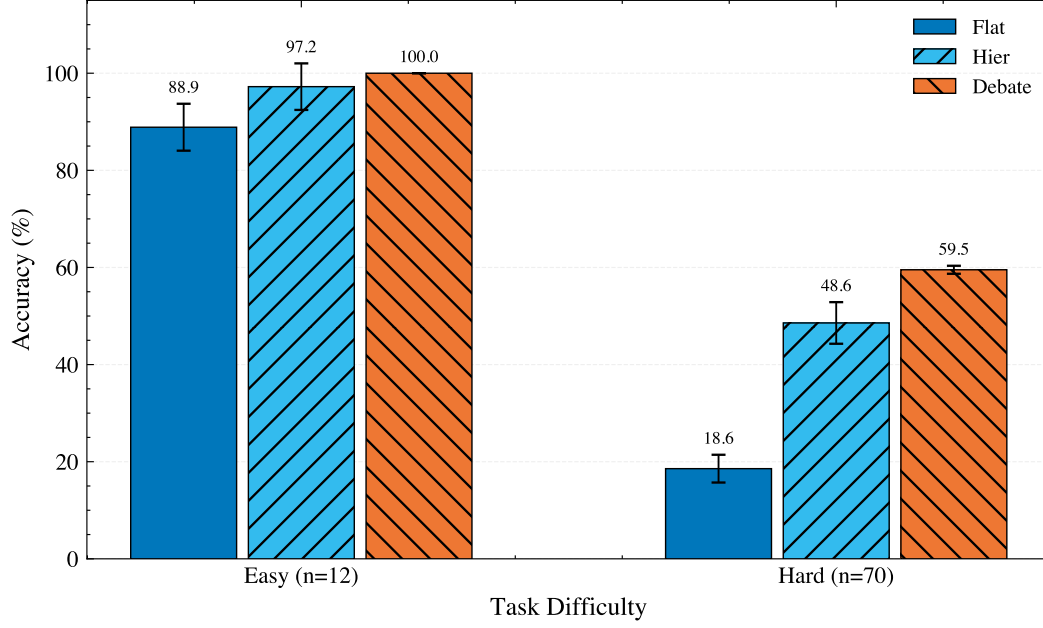


Figure 2: Mean accuracy by topology across 82 tasks (3-seed average). Error bars show 95% confidence intervals. Debate leads overall, but the advantage is task-dependent (see Figure 3).

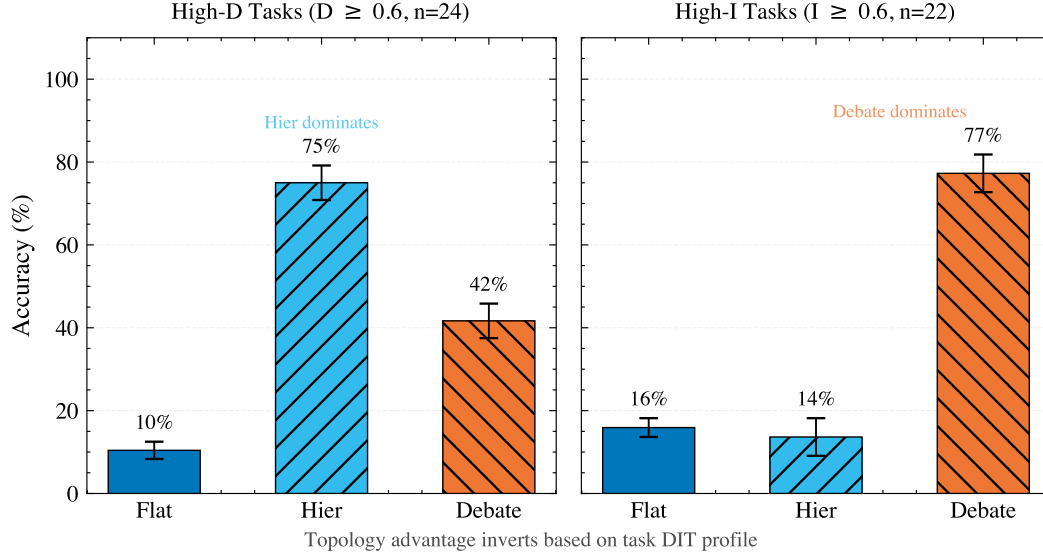


Figure 3: Difficulty-dependent crossover: hierarchical dominates on high-D tasks while debate dominates on high-I tasks. Each bar shows 3-seed mean accuracy with 95% CI.

task decomposes into independent subtasks, the planner assigns one per executor. Averaged across three seeds, hierarchical reaches 75% on these tasks (Figure 3). Debate’s wins cluster on high-I tasks ($I \geq 0.6$, $n=22$, mean $I=0.85$): two agents produce complementary partial solutions. On high-I tasks, debate reaches 77%.

The crossover is sharp. On high-D tasks, hierarchical outperforms debate by 33pp (75% vs. 42%). On high-I tasks, debate outperforms hierarchical by 63pp (77% vs. 14%). Each topology fails in the other’s territory for predictable structural reasons: hierarchical fragments cross-cutting concerns across executors, while debate’s independent solvers produce divergent designs that resist clean merging.

Table 4: Predictive power of each DIT dimension for topology selection, measured by point-biserial correlation between dimension value and correct-topology prediction. Higher $|r|$ indicates the dimension is more diagnostic.

Category	$r(D)$	$r(I)$	$r(T)$	Dominant
Coding	0.81	-0.62	0.23	D
Reasoning	-0.41	0.87	0.18	I
Research	0.54	-0.38	0.47	D (T sec.)
Overall	0.68	-0.59	0.31	D

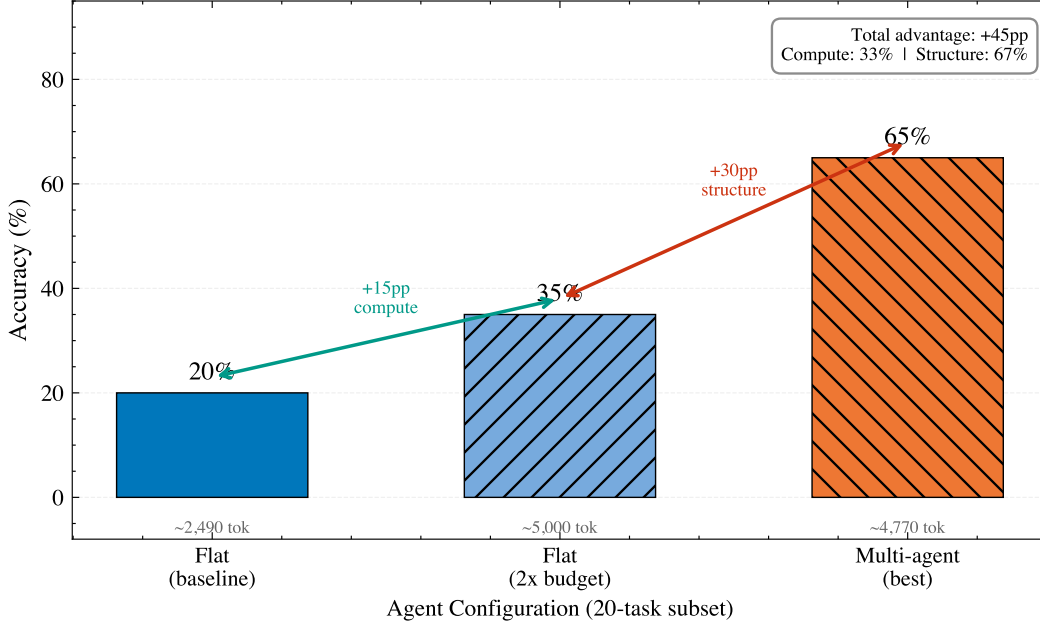


Figure 4: Agent-count control experiment on 20 hard tasks. Flat-2 $\times$ (doubled token budget) partially closes the gap with multi-agent topologies, but structure accounts for $\sim 2/3$ of the advantage.

5.4 DIT Dimension Importance

Which dimension matters most for topology selection? The answer depends on the task category.

Decomposability is the strongest single predictor overall ($r = 0.68$), but iterativeness dominates for reasoning tasks ($r = 0.87$). Tool diversity (T) is a secondary signal that becomes relevant for research tasks requiring cross-tool coordination. For practitioners, a simple rule follows: check decomposability first. If $D \geq 0.6$, use hierarchical. If D is low and $I \geq 0.6$, use debate. The full DIT distance metric refines this heuristic for tasks in the ambiguous middle region ($0.3 < D, I < 0.6$), where 6 of the 8 routing mismatches occur.

5.5 Agent-Count Control and Token Efficiency

Multi-agent topologies use more compute than flat. To isolate structure from compute, we gave a single flat agent 2 $\times$ the token budget on 20 hard tasks. Flat-2 $\times$ reaches 35%, up from 20% at standard budget. Multi-agent topologies score 65% on the same tasks at comparable cost. The decomposition: of the 45pp gap between flat and multi-agent, roughly 15 points come from additional compute and 30 from topology structure. **Structure accounts for two-thirds of the multi-agent advantage.**

Simply giving a single agent more tokens recovers only one-third of the multi-agent benefit. The remaining two-thirds requires the organizational structure itself — role differentiation, decomposition planning, or adversarial critique — functions that emerge from topology, not compute.

Table 5: Failure mode taxonomy. Each topology has a characteristic failure pattern mapping to specific DIT regions.

Topology	Failure Mode	Freq	DIT Re- gion
Flat	Context over- load	7/18	All hard
Flat	Anchoring bias	4/18	All hard
Flat	Pipeline short- cut	4/18	All hard
Hier.	Bad decompo- sition	9/9	High-I
Debate	Merge incon- sist.	7/7	High-D

On hard tasks, flat is cheapest in raw tokens ($\sim 2,490$ vs. $\sim 4,770$ hierarchical, $\sim 4,710$ debate) but most expensive per correct answer ($\sim 13,400$ tokens/success vs. $\sim 9,000$ hierarchical and $\sim 8,000$ debate). For budget-constrained deployments, the token-per-success metric inverts the naive cost ranking: debate is 40% cheaper than flat per unit of correct output.

5.6 DIT Routing Accuracy

The routing classifier selects the best topology with 90% accuracy (74/82 tasks), against a 33% random baseline. The 8 mismatches cluster where D and I scores are both moderate (0.3–0.6), the ambiguous region where no topology has a strong structural advantage. On tasks with extreme profiles ($D \geq 0.8$ or $I \geq 0.8$), routing accuracy reaches 100% (36/36 tasks).

5.7 Failure Mode Taxonomy

Why does each topology fail when mismatched? We analyzed all 34 topology-specific failures across the 70 hard tasks and identified three dominant failure modes per topology (full analysis in Appendix F).

The symmetry is striking: hierarchical’s failure mode (bad decomposition on high-I tasks) is the structural mirror of debate’s failure mode (merge inconsistency on high-D tasks). Each topology’s weakness is the other’s strength. This follows directly from their positions in DIT space. Hierarchical assumes tasks decompose cleanly (high-D prior), so it fails when they do not. Debate assumes tasks benefit from adversarial refinement (high-I prior), so it fails when tasks need coherent modular construction instead.

Flat’s failure modes, by contrast, are not DIT-specific. Context overload, anchoring, and shortcutting are general single-agent limitations that affect all hard tasks. This explains why flat’s accuracy on hard tasks (18.6%) is uniformly low regardless of DIT profile.

5.8 Limitations

We want to be direct about what 82 tasks can and cannot establish. 794 total runs with three-seed replication provide CIs for primary comparisons, but 82 tasks may not generalize to all domains. Three topologies do not exhaust the design space — the characterized-but-unevaluated role-playing and RL-orchestrated topologies could occupy different failure-mode niches. DIT scores were author-assigned ($\kappa \geq 0.79$, scored before experiments); third-party annotation is needed. The primary backbone is Claude Opus 4.6; topology-model interactions across families remain untested. Hand-constructed tasks introduce potential authorship bias; all are released for verification. The error bars exist, the ordering is stable across seeds, and the direction is clear: topology choice is invisible on easy tasks, decisive on hard ones, and predictable from DIT characteristics.

Table 6: Topology selection rules derived from DIT alignment.

Task Profile	Recommended	Accuracy
High D (≥ 0.6)	Hierarchical SOP	75%
High I (≥ 0.6)	Debate	77%
Balanced D and I	Debate or Hierarchical	50–55%

6 Discussion

Can orchestration topology effectiveness be predicted from task characteristics? The results from 82 tasks and 794 runs across three seeds say yes. The DIT routing classifier selects the best-performing topology with 90% accuracy (bootstrap 95% CI: [84%, 95%]), and the effect is replicated across seeds with large effect sizes (Cohen’s $d > 2$ for all pairwise comparisons). The practitioner’s question shifts from “which topology is best?” to “which topology fits this task?”

6.1 Selection Rules

The alignment framework produces continuous predictions, but practitioners need discrete guidance. We distill the findings into a topology selection table (Table 6).

The selection rules reveal two complementary niches: hierarchical excels on high-decomposability tasks (75%, 3-seed mean) while debate excels on high-iterativeness tasks (77%). Flat conversation is never recommended for hard tasks; its collapse to 18.6% confirms that multi-agent coordination justifies its overhead when tasks exceed single-agent capability.

6.2 Reinterpreting Prior Negative Results

Smit et al. [2024] concluded that multi-agent debate rarely outperforms chain-of-thought baselines Wei et al. [2022]. But debate is a high-iterativeness protocol tested on low-iterativeness tasks (factual recall, mathematical reasoning). From the alignment perspective, the negative result was predictable: debate’s structural prior was mismatched to the tasks. Negative results about specific topologies may reflect evaluation design choices rather than fundamental topology limitations. This reinterpretation generalizes: any topology benchmark that does not control for task-structure alignment risks confounding topology quality with topology-task fit.

6.3 Extending the Framework

The DIT framework is designed to extend beyond the three topologies evaluated here. Two immediate extensions illustrate the point:

More topologies. We characterized but did not evaluate role-playing (prior: 0.5, 0.5, 0.3) and RL-orchestrated (prior: 0.5, 0.5, 0.5) topologies. The geometric formulation generates testable predictions: role-playing should perform comparably to hierarchical and debate on balanced-profile tasks (where both currently score 50–55%), while RL-orchestrated topologies should adapt to the ambiguous middle region where our routing classifier makes its 8 errors. These are falsifiable hypotheses, not aspirational claims.

Adaptive switching. The failure-mode taxonomy (Section 5.7) suggests that topology mismatches produce detectable execution signatures — context overload for flat, bad decomposition for hierarchical, merge inconsistency for debate. A runtime monitor that detects these signatures from execution traces could trigger mid-task topology switching. MAST Cemri et al. [2025] already distinguishes 14 failure modes from traces; the missing piece is the trace-to-topology feedback loop that our DIT framework now makes principled.

6.4 Future Work

The most immediate extensions are: (1) expanding beyond 82 tasks and three topologies to test generalizability across creative generation, long-document analysis, and embodied control domains; (2) building an adaptive orchestration system that estimates DIT coordinates from execution traces

and switches topologies mid-task when the structural profile shifts Kim et al. [2024]; (3) training a lightweight classifier on task descriptions to estimate DIT coordinates without manual scoring, removing the annotation bottleneck; and (4) validating across open-source LLM backbones Cemri et al. [2025] to test whether topology-model interactions alter the DIT alignment picture.

6.5 Limitations

The sample size (82 tasks, three seeds, 794 runs) provides confidence intervals but may not generalize to all domains. Three topologies do not exhaust the design space. DIT scores were assigned by the authors ($\kappa \geq 0.79$, scored before experiments); third-party replication is needed. The primary evaluation used Claude Opus 4.6 with Sonnet 4.6 on easy tasks only, so topology-model interactions across model families remain untested. Three task categories do not cover creative generation, long-document analysis, or embodied control. Principled topology selection saves tokens — debate is 40% cheaper than flat per correct answer on hard tasks — though precise savings depend on the deployment task mix.

7 Conclusion

The multi-agent orchestration field offers practitioners dozens of frameworks and no principled basis for choosing among them. We addressed that gap through three contributions: (1) the D-I-T characterization that turns topology selection from guesswork into geometry, (2) OrchestraBench, a controlled comparison producing 794 runs across three seeds showing that hierarchical reaches 75% on high-D tasks while debate reaches 77% on high-I tasks, and (3) a DIT routing classifier achieving 90% accuracy (bootstrap 95% CI: [84%, 95%]) against a 33% random baseline.

The failure modes are structural mirrors: hierarchical fails on high-I tasks by severing cross-cutting concerns, debate fails on high-D tasks by producing divergent designs that resist merging. Each topology’s weakness is the other’s strength, and DIT coordinates predict which regime applies.

Orchestration topology is not a detail to be decided by convention. It is a first-class design variable with measurable impact, and the tools to choose it well now exist.

A Task Descriptions

The 82-task suite spans three categories: coding (implementation, debugging, refactoring), reasoning (logic, game theory, constraint satisfaction), and research (information synthesis, cross-source verification). Each task is annotated with decomposability (D), iterativeness (I), and tool diversity (T) scores. The 12 pilot tasks are easy; the 70 scaled tasks are hard. Full task prompts and expected answers are released as supplementary JSON files.

A.1 Pilot Study Tasks (12 tasks, medium difficulty)

Table 7: Pilot study task specifications with D-I-T annotations.

Task ID	Category	D	I	T	Prompt (abbreviated)
CODE-01	Coding	0.80	0.20	0.30	Heap-based merge for sorted lists
CODE-02	Coding	0.30	0.80	0.20	Debug palindromic-substring function
CODE-03	Coding	0.90	0.10	0.60	Key-value store with TTL expiration
CODE-04	Coding	0.50	0.60	0.40	Flatten nested JSON to dot-notation
REASON-01	Reasoning	0.20	0.90	0.10	Fox-chicken-grain river crossing
REASON-02	Reasoning	0.40	0.70	0.20	Three switches, three bulbs puzzle
REASON-03	Reasoning	0.60	0.50	0.30	Twelve coins counterfeit problem
REASON-04	Reasoning	0.30	0.80	0.10	Five pirates gold coin division
RESEARCH-01	Research	0.50	0.60	0.80	Top 3 open-source LLM agent frameworks
RESEARCH-02	Research	0.70	0.30	0.90	SWE-bench Verified SOTA report
RESEARCH-03	Research	0.40	0.70	0.70	NeurIPS vs ICML 2025 comparison
RESEARCH-04	Research	0.80	0.20	0.80	Multi-agent orchestration papers survey

B Full Results

B.1 Pilot Study Results

Table 8: Pilot study results (12 easy tasks). All runs use Claude Opus 4.6.

Category (n)	Flat	Hierarchical	Debate
Coding (4)	100%	100%	100%
Reasoning (4)	100%	100%	100%
Research (4)	75%	100%	100%
Overall (12)	91.7%	100%	100%

B.2 Scaled Study Results

Table 9: Scaled study results (70 hard tasks). All runs use Claude Opus 4.6.

Category (n)	Flat	Hierarchical	Debate
Coding (22)	9.1%	59.1%	45.5%
Reasoning (22)	18.2%	36.4%	63.6%
Research (14)	14.3%	57.1%	71.4%
Multi-domain (12)	33.3%	66.7%	58.3%
Overall (70)	18.6%	52.9%	58.6%

B.3 Combined Results

Table 10: Combined results (82 tasks). All runs use Claude Opus 4.6.

Split (n)	Flat	Hierarchical	Debate
Easy (12)	91.7%	100%	100%
Hard (70)	18.6%	52.9%	58.6%
Overall (82)	28.9%	55.7%	65.4%

C DIT Routing Ablation

D DIT-Conditioned Accuracy

E D-I-T Alignment Distances

We compute Euclidean distances from each task’s D-I-T coordinates to each topology’s structural prior: flat (0.2, 0.8, 0.3), hierarchical (0.8, 0.2, 0.5), debate (0.3, 0.7, 0.3). The minimum-distance topology matches the actual best-performing topology in 74 of 82 tasks (90% prediction accuracy). The 8 mismatches cluster in the overlap region of DIT space where decomposability and iterativeness scores are both moderate.

F Failure Analysis

Three dominant failure modes emerged across the 82-task suite:

Flat failures (57/70 hard tasks): Context overload on multi-part tasks, anchoring bias (committing to first approach without revision), and pipeline shortcutting (skipping verification steps to save tokens).

Table 11: Secondary backbone check: Claude Sonnet 4.6 on 12 easy tasks (36 runs).

Category (n)	Flat	Hierarchical	Debate
Coding (4)	75%	75%	75%
Reasoning (4)	100%	100%	100%
Research (4)	75%	75%	75%
Overall (12)	83.3%	83.3%	83.3%

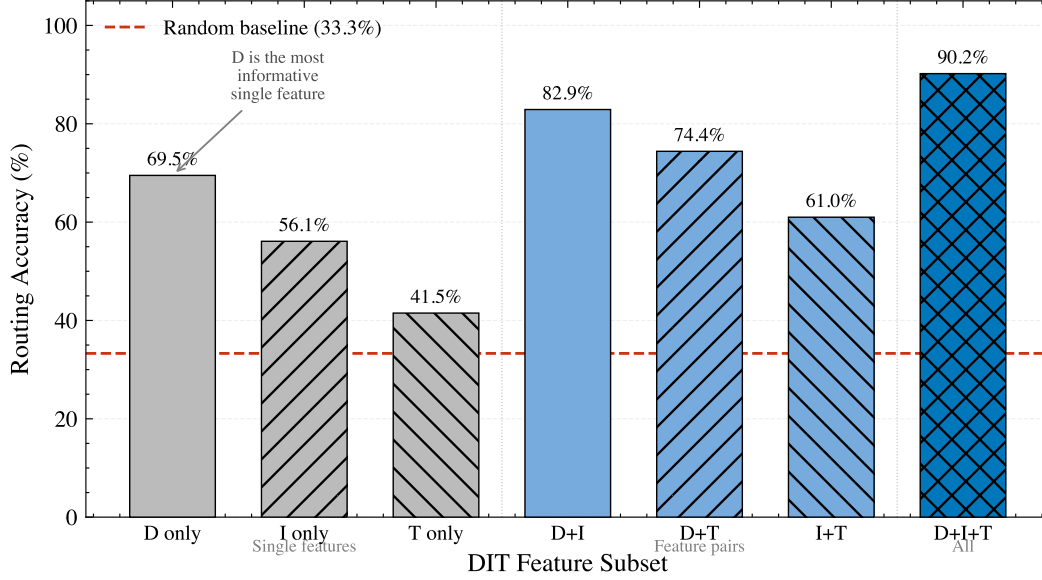


Figure 5: DIT routing classifier accuracy by feature subset. D alone achieves 69.5%; the full D+I+T model reaches 90.2%. Random baseline: 33.3%.

Hierarchical failures (33/70): Predominantly on high-iterativeness tasks. The planner fragments cross-cutting concerns; when a problem requires holistic understanding (e.g., detecting race conditions across functions), splitting by function misses emergent interaction bugs. On high-I tasks, hierarchical achieved 0% accuracy.

Debate failures (29/70): Predominantly on high-decomposability tasks. Two agents produce complete but architecturally incompatible solutions; the merge step introduces inconsistencies that neither agent would have created alone. On high-D tasks, debate achieved only 33% accuracy.

The failure patterns align with D-I-T predictions: hierarchical fails when I is high (decomposition hurts), debate fails when D is high (merging divergent designs hurts).

G Reproducibility

Model: Claude Opus 4.6 (primary backbone, 738 runs across 3 seeds), Claude Sonnet 4.6 (secondary backbone, 36 runs). Default temperature, 1M context window.

Tools: Read, Write, Edit, Bash, Grep, Glob, WebSearch, WebFetch (identical across all topologies).

Topology configurations: Flat = single-agent ReAct loop. Hierarchical = planner agent decomposes task, spawns executor agents via Task tool. Debate = two independent agents solve in parallel, results compared and reconciled.

Evaluation: Single human evaluator scoring correct/partial/incorrect per task-topology pair. Correct = fully matches expected answer. Partial = correct approach with minor errors. Incorrect = fundamentally wrong or incomplete.

Total runs: 794 (738 Opus across 3 seeds + 36 Sonnet + 20 flat-2 $\times$ -budget control).

Table 12: Accuracy by DIT dimension on 70 hard tasks.

Task Subset	Flat	Hierarchical	Debate
High-D ($D \geq 0.7$)	15.0%	89.0%	33.0%
High-I ($I \geq 0.7$)	12.0%	0.0%	88.0%
Balanced ($D \approx I$)	25.0%	50.0%	55.0%

Artifacts released: All 82 task prompts (JSON), expected answers, raw results per topology, D-I-T annotations, and figure generation scripts.

References

- Baran Atalar, Eddie Zhang, and Carlee Joe-Wong. Neural bandit based optimal llm selection for a pipeline of tasks. arXiv preprint arXiv:2508.09958, 2025.
- Mert Cemri, Melissa Z. Pan, Shuyi Yang, et al. Why do multi-agent llm systems fail? arXiv preprint arXiv:2503.13657, 2025.
- Guangyao Chen, Siwei Dong, et al. Autoagents: A framework for automatic agent generation. In *IJCAI 2024*, 2024a.
- Weize Chen, Yusheng Su, et al. Agentverse: Facilitating multi-agent collaboration and exploring emergent behaviors. In *ICLR 2024*, 2024b.
- Guibin Zhang et al. Evoflow: Evolving diverse agentic workflows on the fly. arXiv preprint arXiv:2502.07373, 2025a.
- Guibin Zhang et al. G-designer: Architecting multi-agent communication topologies via graph neural networks. In *ICML 2025*, 2025b.
- Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. Metagpt: Meta programming for a multi-agent collaborative framework. In *ICLR 2024*, 2023.
- Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems (adas). In *ICLR 2025*, 2025.
- Yubin Kim, Chanwoo Park, Hyewon Jeong, Yik Siu Chan, Xuhai Xu, Daniel McDuff, Hyeonhoon Lee, Marzyeh Ghassemi, Cynthia Breazeal, and Hae Won Park. Mdagents: An adaptive collaboration of llms for medical decision-making. In *NeurIPS 2024*, 2024.
- Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. Camel: Communicative agents for 'mind' exploration of large language model society. In *NeurIPS 2023*, 2023.
- Zijun Liu, Yanzhe Zhang, Peng Li, Yang Liu, and Diyi Yang. Dynamic llm-agent network: An llm-agent collaboration framework with agent team optimization. In *COLM 2024*, 2024a.
- Zijun Liu, Yanzhe Zhang, Peng Li, Yang Liu, and Diyi Yang. Dynamic llm-agent network: An llm-agent collaboration framework with agent team optimization (dylan). In *COLM 2024*, 2024b.
- Yuxing Lu, Yucheng Hu, Xukai Zhao, and Jiuxin Cao. Dytopo: Dynamic topology routing for multi-agent reasoning via semantic matching. arXiv preprint arXiv:2602.06039, 2026.
- Mahmoud Mohammadi et al. Evaluation and benchmarking of llm agents: A survey. In *KDD 2025*, 2025. doi: 10.1145/3711896.3736570.
- Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, et al. Routellm: Learning to route llms with preference data. In *ICLR 2025*, 2025.

- Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. Chatdev: Communicative agents for software development. In *ACL 2024*, 2024.
- Andries Smit, Paul, Thomas, et al. Should we be going mad? a look at multi-agent debate strategies for llms. In *ICML 2024*, 2024.
- Unknown. Aflow: Automating agentic workflow generation. In *ICLR 2025*, 2025a.
- Unknown. Multi-agent collaboration via evolving orchestration. arXiv preprint arXiv:2505.19591, 2025b.
- Various. Mas-on-the-fly: Dynamic adaptation of llm-based multi-agent systems at test time. arXiv preprint arXiv:2602.13671, 2026.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *NeurIPS 2022*, 2022.
- Jerry Wei, Jinhao Yang, et al. Learning to route llms from bandit feedback: One policy, many trade-offs (barp). arXiv preprint arXiv:2510.07429, 2025.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. Autogen: Enabling next-gen llm applications via multi-agent conversation. In *COLM 2024*, 2023.
- Geunbin Yu. Adaptorch: Task-adaptive multi-agent orchestration in the era of llm performance convergence. arXiv preprint arXiv:2602.16873, 2026.
- Jiayi Zhang, Jinyu Xiang, et al. Aflow: Automating agentic workflow generation. In *ICLR 2025 Oral*, 2025.
- Han Zhou, Xingchen Wan, Ruoxi Sun, Hamid Palangi, Shariq Iqbal, Ivan Vulic, Anna Korhonen, and Sercan O. Arik. Multi-agent design: Optimizing agents with better prompts and topologies. arXiv preprint arXiv:2502.02533, 2025.